

Implementation

Phase 1

Event Structure (event.h)

- 64 bytes, `#pragma pack(1)` for tight packing
- `uint64_t nano_stamp` : nanosecond timestamp
- `uint64_t sequence_number` : event identifier
- `uint64_t payload_size` : payload byte count
- `Handle payload_handle {}` : memory pool handle (index + generation)
- `uint8_t payload(32)` : embedded payload buffer
- `static_assert(sizeof(Event) == 64)`

RingBuffer (ring_buffer.h)

- Template: `RingBuffer<uint32_t Capacity>` — Capacity must be power of two
- `Event buffer(Capacity)` : fixed-size circular buffer
- `std::atomic<uint32_t> write_index` : producer position
- `std::atomic<uint32_t> read_index` : consumer position
- `static constexpr uint32_t MASK = Capacity - 1` : bitwise wrapping
- Methods:
 - `bool push(const Event&)` : write event, release semantics, returns false if full
 - `bool pop(Event&)` : read event, acquire semantics, returns false if empty
 - `bool is_empty() const` : check if buffer has no events
 - `bool is_full() const` : check if buffer cannot accept events
 - `uint32_t size() const` : return count of events in buffer
- Index wrapping uses bitwise AND & MASK (one CPU cycle)
- Full check: `(next_write & MASK) == read_idx`
- Empty check: `write_idx == read_idx`
- `static_assert` enforces power-of-two capacity at compile time

MemoryPool (memory_pool.h)

- Struct: `Handle` with index and generation fields
- Template: `MemoryPool<typename T, uint32_t Capacity>`
- `std::array<T, Capacity> storage` : pre-allocated objects
- `std::array<uint32_t, Capacity> generations` : version counter per slot
- `std::array<bool, Capacity> available` : free slot tracking (default filled with true)
- Methods:
 - `Handle allocate()` : find first available slot, increment generation, mark unavailable, return `Handle`
 - `void deallocate(Handle)` : validate generation matches, mark slot available
 - `T* get(Handle)` : validate generation matches, return pointer to object or nullptr
- Generation validation catches use-after-free at runtime

EventBus (event_bus.h)

- Template: `EventBus<uint32_t BufferCapacity, uint32_t PayloadCapacity>`
- `RingBuffer<BufferCapacity> command_buffer` : inbound events
- `RingBuffer<BufferCapacity> response_buffer` : outbound events
- `MemoryPool<uint8_t, PayloadCapacity> payload_pool` : payload storage
- `uint32_t latency_ns` : fixed delay simulation
- `uint32_t jitter_ns` : random delay variation
- `float packet_loss_rate` : drop probability (0.0 to 1.0)
- Methods:

- `void send(const Event&) : apply latency/jitter/loss, write to command buffer`
- `bool receive(Event&) : apply latency/jitter/loss, read from response buffer`
- `void push(const Event&) : write event to command buffer`
- `Handle allocate_payload()` : allocate slot in payload pool
- `bool pop(Event&) : read event from command buffer, returns false if empty`
- `uint8_t* get_payload(Handle) : retrieve payload pointer`
- `void deallocate_payload(Handle) : mark slot available for reuse`
- Payload embedded directly in Event structure (40 bytes)
- Latency and jitter simulation uses `<random>` and `std::this_thread::sleep_for()`

Phase 1 Performance Results

- **Latency:** 3.7µs median, 26ns minimum, 95µs P99 (1M events, 65KB buffer)
- **Throughput:** 13.6M events/sec (target: 1M/sec, exceeds by 13x)
- **Distribution:** 33% of events complete sub-1µs, 56% sub-10µs, 99% sub-100µs
- **Benchmark:** 1 million events, power-of-two 65536-byte buffer, concurrent producer/consumer
- Note: 500ns target requires CPU affinity and dedicated cores (deferred to Phase 2)

```

Number of Events: 1000000.
Buffer Size: 1024*64 bytes.

Average latency: 19960.2 ns
Min latency: 26 ns
Max latency: 243530 ns

P50 (median): 3767 ns
P90: 59373 ns
P95: 69157 ns
P99: 94893 ns
P99.9: 220587 ns

Throughput: 1.35643e+07 events/sec
Total time: 0.0737232 seconds

```

Phase 1 Complete

- Lock-free SPSC ring buffer with acquire/release semantics
- Power-of-two capacity with bitwise AND masking
- Zero dynamic allocation in critical path
- Comprehensive performance benchmark with percentile analysis
- Ready for Phase 2 integration

Phase 2: Hardware Isolation and Payload Integration

CPU Affinity (affinity.hpp)

- Header: `src/core/affinity.hpp`
- Template function: `bind_thread_to_core(ThreadType& thread, int core_id)`
- Extracts native handle via `thread.native_handle()`
- Creates `cpu_set_t`, zeros with `CPU_ZERO()`, sets core with `CPU_SET()`
- Calls POSIX `pthread_setaffinity_np()` with handle, size, and CPU set
- Throws `std::runtime_error` on invalid core ID (negative) or binding failure
- Supports both `std::thread` and `std::jthread`
- Binding is immediate at thread startup

Publisher (publisher.hpp)

- Class template: `Publisher<typename EventBus>`
- `std::jthread worker_` : owned worker thread
- `EventBus& event_bus_` : reference to event bus (dependency injection)
- `std::atomic<bool> shutdown_flag_` : graceful shutdown signal

- `std::condition_variable notify_cv_` : wake worker on demand
- Methods:
 - `start()` : launch worker thread pinned to core 1
 - `ready()` : notify condition variable to wake worker
 - `wait()` : set shutdown flag, notify worker, join thread
- Work loop: allocate payload, get pointer, write data, create event with handle, push to ring buffer
- Move semantics supported; copy operations deleted
- Destructor calls `wait()` for safe cleanup

Subscriber (subscriber.hpp)

- Class template: `Subscriber<typename EventBus>`
- `std::jthread worker_` : owned worker thread
- `EventBus& event_bus_` : reference to event bus (dependency injection)
- `std::atomic<bool> shutdown_flag_` : graceful shutdown signal
- `std::condition_variable notify_cv_` : wake worker on demand
- Methods:
 - `start()` : launch worker thread pinned to core 2
 - `ready()` : notify condition variable to wake worker
 - `wait()` : set shutdown flag, notify worker, join thread
- Work loop: pop event, extract handle, resolve handle to payload, read data, deallocate handle
- Move semantics supported; copy operations deleted
- Destructor calls `wait()` for safe cleanup

Phase 2 Complete

- Publisher and subscriber owned threads with graceful shutdown.
- Memory pool integrated: allocate → embed → resolve → deallocate.
- Affinity binding tested and confirmed on cores 1 and 2.
- Ready for Phase 3.

Phase 3: Ingress Pipeline & Memory Safety

Abstract Ingress Interface (IngressReceiver)

- Header: `src/ingress/ingress.h`
- Pure virtual class: `IngressReceiver`
- Methods:
 - `virtual size_t poll(uint8_t* destination_buffer, size_t max_len) = 0` : non-blocking poll for raw bytes, returns byte count read
- Contract: no heap allocation, caller provides buffer, returns 0 if no data

Mock Ingress Source (MockIngressSource)

- Header: `src/ingress/mock_ingress.h`
- Class: `MockIngressSource` : `public IngressReceiver`
- Members:
 - `std::vector<std::vector<uint8_t>> pregenerated_frames_` : 1000 deterministic 128-byte order messages
 - `size_t current_frame_index_` : current frame pointer
 - `size_t current_offset_` : offset within frame
- Methods:
 - Constructor: pre-generates 1000 synthetic frames (sequence, timestamp, order_id, side, price, quantity, symbol)
 - `size_t poll(uint8_t* buffer, size_t max_len)` override : return frame bytes sequentially, no allocation
- Behavior: deterministic, reproducible, EOF after 1000 frames

Event Director (EventDirector)

- Header: `src/ingress/event_director.h`
- Class template: `EventDirector<typename EventBus>`

- Members:
 - `IngressReceiver& ingress_` : dependency injection
 - `EventBus& event_bus_` : reference to event bus
 - `uint8_t receive_buffer_(4096)` : pre-allocated stack buffer
 - `uint64_t sequence_counter_` : monotonic event ID
 - `std::atomic<bool> shutdown_flag_` : graceful shutdown signal
 - `std::jthread worker_` : owned worker thread pinned to core 1
 - `std::vector<uint64_t> work_latencies_` : per-event ingestion latency
- Methods:
 - `void start()` : launch worker, bind to core 1
 - `void wait()` : set shutdown flag, join thread
 - `uint64_t sequence_count() const` : return events processed
 - `const std::vector<uint64_t>& get_latencies() const` : return latency vector
- Work loop: poll ingress → allocate payload → copy bytes → create Event → push to ring buffer
- Polling: busy-loop with `_mm_pause()` on empty reads (no OS sleep)

Free-List Memory Pool (FreeListPool)

- Header: `src/memory-pool/free_list_pool.h`
- Class template: `FreeListPool<typename T, uint32_t Capacity>`
- Members:
 - `std::array<T, Capacity> storage_` : pre-allocated objects
 - `std::array<uint32_t, Capacity> generations_` : version counter per slot
 - `std::array<uint32_t, Capacity> free_stack_` : LIFO stack of free indices
 - `uint32_t free_count_` : number of available slots
- Methods:
 - `Handle allocate()` : pop from `free_stack`, increment generation, return Handle — $O(1)$
 - `void deallocate(Handle)` : validate generation, push index back — $O(1)$
 - `T* get(Handle)` : validate generation, return pointer — $O(1)$
- Replaces linear-search `MemoryPool` for $O(N) \rightarrow O(1)$ allocation performance

Guarded Memory Pool (GuardedMemoryPool)

- Header: `src/guarded-pool/guarded_memory_pool.hpp`
- Class template: `GuardedMemoryPool<typename T, uint32_t Capacity>`
- Allocation: `mmap` pool + one unmapped guard page at end
- Protection: `mprotect(guard_page, PROT_NONE)` removes all access
- Methods:
 - Constructor: allocate page-aligned memory, apply `mprotect`
 - Destructor: `munmap` cleanup
 - `T* get(uint32_t index)` : return pointer (or guard page if out of bounds)
- Hardware bounds checking: any write past `Capacity` triggers SIGSEGV (zero runtime overhead)

Phase 3 Performance Results

Before optimizations:

- Ingestion latency: P50 56ns, P95 248ns, P99 531ns
- Throughput: 10.9K events/sec

After FreeListPool (O(1) allocation):

- Ingestion latency: P50 33ns, P95 173ns, P99 336ns
- Throughput: 12.26K events/sec
- Improvement: P50 –41%, P95 –30%, P99 –37%

After `_mm_pause()` busy-poll (no OS sleep):

- Ingestion latency: P50 24ns, P95 25ns, P99 35ns

- Throughput: 94.5K events/sec
- Improvement: P50 -27%, P95 -85%, P99 -90%, throughput +671%

Final metrics (WSL-constrained; bare metal ~1M events/sec):

- Ingestion latency: P50 24ns, P95 25ns, P99 35ns (sub-microsecond)
- Throughput: 94.5K events/sec
- Memory allocation: O(1) via free-list stack
- Guard page: zero nanosecond overhead, hardware-enforced bounds

CPU Core Binding

- EventDirector: Core 1 (producer, isolated from OS)
- Subscriber: Core 2 (consumer, isolated from OS)
- Core 0: Reserved for OS interrupts and background tasks
- Cores 3+: Available for future pipeline stages (Matching Engine, etc.)

Optimization Decisions

- **Busy-poll with `_mm_pause()`** : Eliminates OS nanosleep syscalls on isolated cores. CPU hint prevents pipeline thrashing. Only for tight ingestion loops; use OS sleep for tests and initialization.
- **Free-list allocation**: LIFO stack replaces linear search. O(1) pop/push in 2-5ns. Eliminates tail-latency spikes (P99 regression).
- **Ring buffer capacity**: Kept at 8192 (fits comfortably in L2/L3 cache). Larger buffers do not reduce steady-state latency; only absorb backpressure if consumer stalls.
- **Memcpy alignment**: Not optimized. Stack and pool allocations are naturally cache-line aligned by compiler. Measurable gain unlikely without real network I/O.
- **MockIngressSource frame size**: Kept at 128 bytes (realistic financial wire format). Increasing batch size inflates throughput figures without representing true domain protocol.

Phase 3 Complete

- Abstract ingress interface enables pluggable sources (mock, socket, DPDK, file)
- Mock source provides deterministic, reproducible test data
- EventDirector orchestrates poll-allocate-copy-push pipeline with sub-microsecond latency
- FreeListPool eliminates O(N) allocation spikes; allocation now O(1) 2-5ns
- GuardedMemoryPool detects buffer overflow via hardware segmentation fault
- Busy-poll with `_mm_pause()` eliminates OS scheduler latency on isolated core
- Ready for Phase 4 (Matching Engine)